

Projekt indywidualny - Wirtualizacja w systemie FreeBSD na poziomie systemu operacyjnego z wykorzystaniem więzień na maszynie bezdyskowej

Szymon Skiba 319098

16 czerwca 2023

1 Abstrakt

Jedną z technologii wirtualizacyjnych dostępnych w systemie FreeBSD są więzienia (jails). Więzienia stanowią środowisko izolowane, które pozwala na uruchamianie niezależnych instancji systemu FreeBSD na jednym fizycznym serwerze. Wykorzystując więzienia, można stworzyć odseparowane przestrzenie działania, w których aplikacje i usługi mogą działać niezależnie od siebie, zachowując jednocześnie wysoki poziom bezpieczeństwa i izolacji.

Celem tego projektu indywidualnego jest zgłębienie tematyki wirtualizacji na poziomie systemu operacyjnego w systemie FreeBSD przy użyciu więzień i stworzenie programu zarządzających środowiskiem więzień jail-manager. Projekt skupia się na zrozumieniu architektury oraz praktycznym wykorzystaniu w tworzeniu wirtualnych środowisk FreeBSD na maszynie bezdyskowej. Wirtualizacja na ramdysku to technika, w której więzienie jest tworzone na dysku RAM, co pozwala na jeszcze większą separację i izolację aplikacji od reszty systemu. W pracy zostaną przedstawione najważniejsze zagadnienia związane z więzieniami FreeBSD, omówiona zostanie obsługa sieci oraz system plików ZFS wykorzystany w infrastrukturze stworzonego programu jail-manager.

W kolejnych etapach zostaną omówione technologie i przykłady ich użycia, które pozwoliły stworzyć skrypty do zarządzania więzieniami - jail-manager

Słowa kluczowe: FreeBSD, więzienia, ZFS

2 Dyskowy system plików - ZFS

ZFS (Zettabyte File System) - to system plików i otwartoźródłowy program do zarządzania nim oraz dyskami. Jest natywny dla FreeBSD. Jego główne cechy determinujące jego użyteczność w procesie zarządzania więzieniami:

- Wprowadza dodatkową warstwę abstrakcji, która odziela procesu zarządzania od fizycznych dysków. Umożliwia tworzenie puli dyskowych (pool) z wirtualnych urządzeń stworzonych z fizycznych dysków.
- Niezawodność i ochrona danych: ZFS wykorzystuje sumy kontrolne (checksums), które automatycznie wykrywają i naprawiają błędy bitowe.
- Dynamiczne zarządzanie przestrzenią dyskową: ZFS pozwala na dynamiczne tworzenie, rozszerzanie i zmniejszanie woluminów bez konieczności wcześniejszej alokacji całej przestrzeni dyskowej.
- Szybkość i wydajność: ZFS wykorzystuje inteligentne mechanizmy buforowania, takie jak Adaptive Replacement Cache (ARC) i Intent Log (ZIL), co przyspiesza operacje odczytu i zapisu danych. Wykorzystuje również pamięć RAM jako pamięć podręczną co dodatkowo przyspiesza działanie.
- Punkty kontrolne (snapshots): ZFS umożliwia tworzenie migawek danych w sposób natychmiastowy i o niskim zużyciu zasobów systemowych.
- Możliwość klonowania zbiorów danych.

2.1 Tworzenie RAM-dysku

RAM dyski charakteryzują się dużą szybkością i wydajnością są dobrym rozwiązaniem w zastosowaniu przy stacji bezdyskowej i w procesie tworzenia więzień. Do tworzenia ramdysku wykorzystałem natywną dla systemów BSD komendę `mdconfig`. `-t malloc` określa typ dysku jako alokowany z pamięci RAM.

Listing 1: Przydzielanie nowego dysku w pamięci o rozmiarze 2 gigabajty (s9.volt.zet.pw.edu.pl/FreeBSD)

```
1 # mdconfig -a -t malloc -s 2g
2 md1
```

2.2 Pula dyskowa

Korzystając z oficjalnej dokumentacji FreeBSD [1] przeanalizowałem tworzenie puli dyskowych ZFS. Pula (pool) to podstawowa jednostka organizacyjna w ZFS. Jest to zestaw jednego lub więcej dysków fizycznych (lub innych urządzeń pamięci masowej) połączonych w celu tworzenia jednego dużego pojemnika na dane.

2.2.1 Tworzenie

Listing 2: Tworzenie prostej puli zfs z jednego dysku (s9.volt.zet.pw.edu.pl/FreeBSD)

```
1 # zpool create disk_pool /dev/md1
2 # df -h
3 Filesystem                Size      Used    Avail Capacity  Mounted on
4 /dev/md0                   484M      217M    228M     49%      /
5 devfs                      1.0K       0B     1.0K      0%      /dev
6 tmpfs                      5.4G       12K     5.4G      0%      /tmp
7 tmpfs                      5.5G       58M     5.4G      1%      /var
8 tmpfs                      6.7G     1.3G     5.4G     19%      /usr
9 ftp://pub                  3.6T     2.7T    920G     75%      /pub
10 ftp://usr/local/zetis     934G      14G    920G      1%      /usr/local/zetis
11 disk_pool                 1.7G      24K     1.7G      0%      /disk_pool
```

Pula dyskowa `disk_pool` jest dostępna jako system plików.

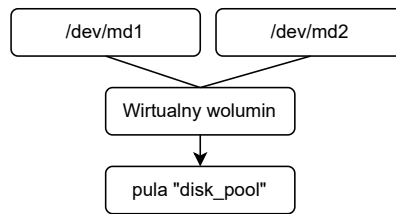
Listing 3: (s9.volt.zet.pw.edu.pl/FreeBSD)

```
1 # mkdir /disk_pool/example
2 # ls /disk_pool
3 example
```

ZFS umożliwia tworzenie puli z dowolnej liczby dysków, przy tworzeniu wystarczy jedynie uwzględnić dodatkowe dyski. Rozmiar puli determinuje wtedy sumaryczny rozmiar użytych dysków

Listing 4: Tworzenie prostej puli zfs z dwóch dysków (s9.volt.zet.pw.edu.pl/FreeBSD)

```
1 # zpool destroy disk_pool
2 # zpool create disk_pool /dev/md1 /dev/md2
3 # df -h
4 Filesystem                Size      Used    Avail Capacity  Mounted on
5 /dev/md0                   484M      217M    228M     49%      /
6 ...
7 disk_pool                 3.6G      24K     3.6G      0%      /disk_pool
8 # zpool status disk_pool
9   pool: disk_pool
10  state: ONLINE
11 config:
12      NAME      STATE    READ WRITE CKSUM
13      disk_pool  ONLINE      0     0     0
14          md1    ONLINE      0     0     0
15          md2    ONLINE      0     0     0
16 errors: No known data errors
```



Rysunek 1: Schemat puli

2.2.2 Tworzenie z montowaniem

Listing 5: Tworzenie prostej puli zfs z jednego dysku z montowaniem (s9.volt.zet.pw.edu.pl/FreeBSD)

```

1 # zpool destroy disk_pool
2 # mkdir /tmp/disk_pool
3 # zpool create -m /tmp/disk_pool disk_pool /dev/md1
4 # df -h
5 Filesystem                Size      Used    Avail  Capacity  Mounted on
6 /dev/md0                   484M      217M    228M    49%      /
7 ...
8 disk_pool                  1.7G       24K    1.7G     0%      /tmp/disk_pool

```

2.2.3 RAIDZ

ZFS umożliwia również łatwą kreację specyficznego dla ZFS mechanizmu redundancji podobnego do RAID-ów o nazwie RAIDZ. RAIDZ wykorzystuje mechanizm rozłożenia danych na poziomie bloku wraz z parzystością, aby rozproszyć dane na kilku dyskach. Pula RAIDZ może tolerować awarię jednego lub więcej dysków, w zależności od konkretnej wersji.

Listing 6: RAIDZ puli zfs z dwóch ram dysków 2g /dev/md2 i /dev/md3(s9.volt.zet.pw.edu.pl/FreeBSD)

```

1 # zpool create raid_pool raidz /dev/md2 /dev/md3
2 # df -h
3 Filesystem                Size      Used    Avail  Capacity  Mounted on
4 /dev/md0                   484M      217M    228M    49%      /
5 ...
6 disk_pool                  1.7G       24K    1.7G     0%      /disk_pool
7 raid_pool                  1.7G       24K    1.7G     0%      /raid_pool
8 # zpool status raid_pool
9   pool: raid_pool
10  state: ONLINE
11 config:
12      NAME      STATE    READ WRITE CKSUM
13      raid_pool  ONLINE    0     0     0
14          raidz1-0  ONLINE    0     0     0
15              md2    ONLINE    0     0     0
16              md3    ONLINE    0     0     0
17 errors: No known data errors

```

2.2.4 Pule lustrzane

ZFS oferuje również wydajniejszą formę konfiguracji puli - pula lustrzana w której dane są duplikowane na osobnych dyskach, tworząc dokładną kopię każdego fragmentu danych.

Listing 7: Pula lustrzana z dwóch ram dysków 2g /dev/md4 i /dev/md5 (s9.volt.zet.pw.edu.pl/FreeBSD)

```

1 # zpool create mir_pool mirror /dev/md4 /dev/md5
2 # df -h
3 Filesystem                Size      Used    Avail  Capacity  Mounted on
4 /dev/md0                   484M      217M    228M    49%      /
5 ...
6 disk_pool                  1.7G       24K    1.7G     0%      /disk_pool
7 raid_pool                  1.7G       24K    1.7G     0%      /raid_pool

```

```

8 mir_pool          1.7G      24K      1.7G      0%      /mir_pool
9 # zpool status mir_pool
10   pool: mir_pool
11   state: ONLINE
12 config:
13     NAME      STATE      READ  WRITE CKSUM
14     mir_pool   ONLINE      0     0     0
15     mirror-0   ONLINE      0     0     0
16     md4        ONLINE      0     0     0
17     md5        ONLINE      0     0     0
18 errors: No known data errors

```

2.2.5 Podsumowanie

Ostatecznie w skrypcie zarządzającym więzzeniami **jail-manager** wykorzystałem tylko mechanizm prostego tworzenia puli ze względu na charakterystykę docelowego środowiska jako stacji bezdyskowej, gdzie mechanizm puli lustrzanych i raidz nie są potrzebne.

2.3 Zbiory danych ZFS

W tej części wykorzystując stworzone pulę będę tworzyć zbiory danych zfs. Zbiór danych jest logicznym kontenerem wewnątrz puli, w którym przechowywane są rzeczywiste dane.

2.3.1 Tworzenie

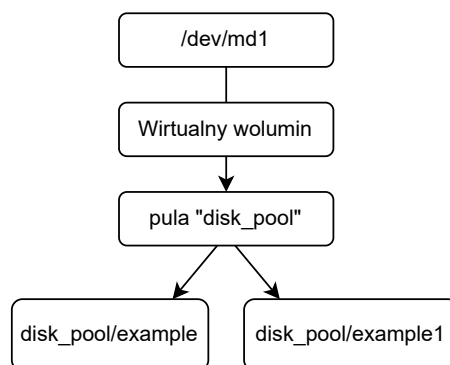
Poniższy przykład demonstruje tworzenie zbiorów danych z ograniczonym dostępnej pamięci dla danego zbioru. Pokazuje również, że oba zbiory danych mogą mieć tę samą ilość dostępnej pamięci, ponieważ zaciągają z tej samej puli i wykorzystują tylko potrzebną pamięć co eliminuje potrzebę partycji i woluminów. Mechanizm ten stosuje później przy kreacji więzień.

Listing 8: Tworzenie zbioru danych z puli `disk_pool` (Listing 5) (s9.volt.zet.pw.edu.pl/FreeBSD)

```

1 # zfs create disk_pool/example
2 # zfs set quota=2G disk_pool/example
3 # zfs create disk_pool/example1
4 # zfs set quota=2G disk_pool/example1
5 # df -h
6 Filesystem      Size    Used   Avail Capacity  Mounted on
7 /dev/md0         484M    217M    228M     49%      /
8 ...
9 disk_pool        1.7G     26K    1.7G      0%      /tmp/disk_pool
10 disk_pool/example 1.7G     24K    1.0G      0%      /tmp/disk_pool/example1
11 disk_pool/example1 1.7G     24K    1.0G      0%      /tmp/disk_pool/example

```



Rysunek 2: Schemat zbiorów danych

2.3.2 Migawki

Polecenie `zfs snapshot` tworzy kopię całego zbioru o dostępie tylko do odczytu. Są bardzo szybkie, ale pozwalają na tworze kopie tylko całego zbioru nie jego części.

Listing 9: Tworzenie migawkami z zbioru danych (Listing 8) (s9.volt.zet.pw.edu.pl/FreeBSD)

```
1 # zfs snapshot disk_pool/example@template
```

Listing 10: Zmiana nazwy obecnej migawki i utworzenie nowej (s9.volt.zet.pw.edu.pl/FreeBSD)

```
1 # zfs rename -r disk_pool/example@template old_snap
2 # zfs snapshot disk_pool/example@template
3 # zfs list -t snapshot -r disk_pool
4 NAME                                USED    AVAIL    REFER    MOUNTPOINT
5 disk_pool/example@old_snap          0B      -        24K     -
6 disk_pool/example@template          0B      -        24K     -
```

Listing 11: Powrót do poprzedniej migawki (s9.volt.zet.pw.edu.pl/FreeBSD)

```
1 # zfs rollback -R disk_pool/example@old_snap
2 # zfs list -t snapshot -r disk_pool
3 NAME                                USED    AVAIL    REFER    MOUNTPOINT
4 disk_pool/example@old_snap          0B      -        24K     -
5 # zfs rename -r disk_pool/example@old_snap template
```

2.3.3 Klonowanie

Listing 12: Klonowanie z migawki (Listing 11) (s9.volt.zet.pw.edu.pl/FreeBSD)

```
1 # zfs clone disk_pool/example@template disk_pool/example2
2 # df -h
3 Filesystem      Size    Used    Avail  Capacity  Mounted on
4 /dev/md0         484M    217M    228M    49%      /
5 ...
6 disk_pool        1.7G     26K    1.7G     0%      /tmp/disk_pool
7 disk_pool/example 1.7G     24K    1.7G     0%      /tmp/disk_pool/example
8 disk_pool/example1 1.7G     24K    1.7G     0%      /tmp/disk_pool/example1
9 disk_pool/example2 1.7G     24K    1.7G     0%      /tmp/disk_pool/example2
```

2.3.4 Podsumowanie

Migawki w ZFS są użyteczne w tworzeniu więzień w FreeBSD, ponieważ umożliwiają tworzenie punktów kontrolnych stanu danych, łatwe przywracanie więzień do wcześniejszego stanu, szybkie tworzenie migawek, wersjonowanie i śledzenie historii oraz dają możliwość szybkiego klonowania całych zbiorów danych co jest użyteczne w kreacji więzień.

2.4 Obsługa sieci w środowisku więzień

Wyróżniającą cechą więzień na tle innych sposobów wirtualizacji jest obsługa wirtualnego stosu sieciowego (VNET), dzięki któremu każde z więzień posiada oddzielny niezależny stos sieciowy. Aby zapewnić tą funkcjonalność należy uruchomić więzień z odpowiednią konfiguracją poleceniem `jail` które wchodzi w skład pakietu FreeBSD Jail, jest narzędziem natywnym dla FreeBSD i odpowiedzialnym za uruchamianie i zatrzymywanie więzień.

2.4.1 Konfiguracja

Omówieni konfiguracja środowiska i więzień znajduje się w podpunkcie 2.5

2.4.2 Interfejsy TUN/TAP

Interfejsy TUN/TAP są wirtualnymi urządzeniami sieciowymi używanymi w systemach Unix-like. Są one używane do tworzenia wirtualnych interfejsów sieciowych, które mogą odbierać i wysyłać pakiety sieciowe, podobnie jak fizyczne interfejsy sieciowe. Interfejsy TUN działają na warstwie sieciowej (warstwa 3 modelu OSI) i umożliwiają przechwytywanie i przekierowywanie ruchu sieciowego na wyższym poziomie. Interfejsy TUN są używane do tworzenia tuneli punkt-do-punkt, gdzie pakiet IP jest enkapsulowany w inny protokół i wysyłany przez sieć. Interfejsy TAP natomiast działają na 2 warstwie modelu osi i symulują pełne urządzenie Ethernet, włącznie z możliwością przesyłania ramek Ethernet.

2.4.3 VLAN

Przyłączenie jaila do sieci jest możliwe dzięki utworzeniu sieci VLAN i przypisaniu interfejsu VLAN do vnet-jail. Ograniczeniem jest to, że identyfikatory VLAN muszą być unikalne dla każdego interfejsu. Jeśli dwa vnet-jails muszą być przypisane do podinterfejsu VLAN w tej samej sieci VLAN, konieczne będzie użycie dwóch fizycznych interfejsów.

2.4.4 Bridge+epair

Epair to wirtualny intrfejs sieciowy który składa się z pary urządzeń, z których jedno znajduje się w więzieniu a drugie w gospodarzu tworząc połączenie między nimi. Bridge to wirtualny mostek sieciowy łączący sieci w drugiej wartwie modelu OSI, przekazuje on ruch z jednego interfesju na drugi co przypomina działaniem przełącznik.

Bridge i Epair pozwalają na stworzenie dowolnej liczby więzień i łączenie ich w sieci różnej topologii. W ponniższym przykładzie 3 zademstrowny jest sposób połączenia wykorzystywany w jail-manager.

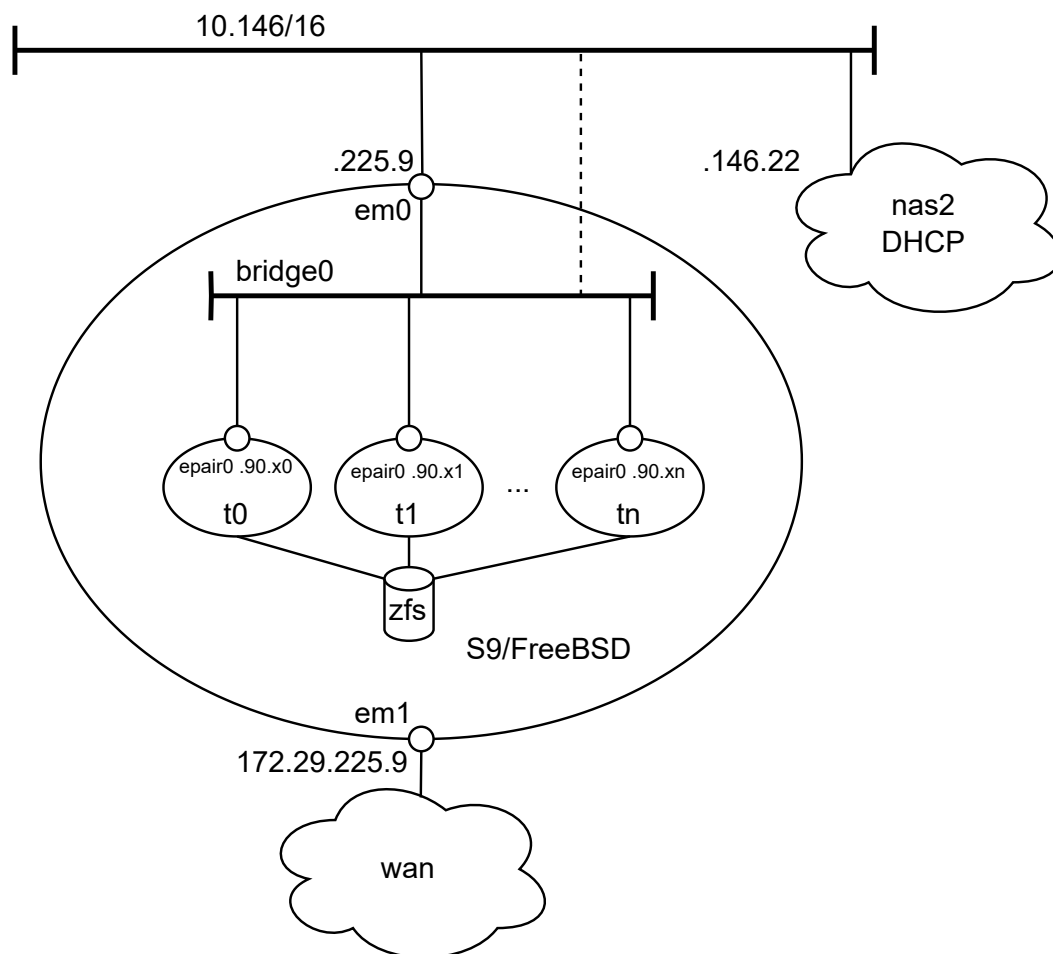
Listing 13: Użycie więzień z epir i bridge (s9.volt.zet.pw.edu.pl/FreeBSD)

```
1 # ifconfig bridge create up
2 bridge0
3 # for i in $(jot 3); do ifconfig epair create up; done
4 epair0a
5 epair1a
6 epair2a
7 # ifconfig bridge0 addm em0 addm epair0a addm epair1a addm epair2a
8 # sh << EOF
9 >for i in $(seq 0 2); do
10 >jail -c name=test$i host.hostname=test$i persist vnet vnet.interface=epair${i}b \
    devfs_ruleset=5
11 >jexec test$i
12 >dhclient epair${i}b
13 >done
14 >EOF
15 DHCPDISCOVER on epair0b to 255.255.255.255 port 67 interval 4
16 DHCPOFFER from 10.146.146.22
17 unknown dhcp option value 0xaf
18 DHCPREQUEST on epair0b to 255.255.255.255 port 67
19 DHCPACK from 10.146.146.22
20 unknown dhcp option value 0xaf
21 bound to 10.146.90.174 -- renewal in 1800 seconds.
22 DHCPDISCOVER on epair1b to 255.255.255.255 port 67 interval 3
23 DHCPOFFER from 10.146.146.22
24 unknown dhcp option value 0xaf
25 DHCPREQUEST on epair1b to 255.255.255.255 port 67
26 DHCPACK from 10.146.146.22
27 unknown dhcp option value 0xaf
28 bound to 10.146.90.175 -- renewal in 1800 seconds.
29 DHCPDISCOVER on epair2b to 255.255.255.255 port 67 interval 8
30 DHCPOFFER from 10.146.146.22
31 unknown dhcp option value 0xaf
32 DHCPREQUEST on epair2b to 255.255.255.255 port 67
33 DHCPACK from 10.146.146.22
34 unknown dhcp option value 0xaf
```

```

35 bound to 10.146.90.70 -- renewal in 1800 seconds.
36 # for i in {0..2}; do echo test$i ; jexec test$i ifconfig | grep inet ; done
37 test0
38     inet 10.146.90.174 netmask 0xffff0000 broadcast 10.146.255.255
39 test1
40     inet 10.146.90.175 netmask 0xffff0000 broadcast 10.146.255.255
41 test2
42     inet 10.146.90.70 netmask 0xffff0000 broadcast 10.146.255.255
43 # ping -c 2 10.146.90.174
44 PING 10.146.90.174 (10.146.90.174): 56 data bytes
45 64 bytes from 10.146.90.174: icmp_seq=0 ttl=64 time=0.323 ms
46 64 bytes from 10.146.90.174: icmp_seq=1 ttl=64 time=0.240 ms
47
48 --- 10.146.90.174 ping statistics ---
49 2 packets transmitted, 2 packets received, 0.0% packet loss
50 round-trip min/avg/max/stddev = 0.240/0.282/0.323/0.041 ms

```



Rysunek 3: Schemat powstałej sieci

2.5 Parametry konfiguracji więzień i środowiska

Proces tworzenia więzień za pomocą pakietu FreeBSD Jail oferuje dużą ilość parametrów i opcji, co pozwala na dużą dowolność, wymaga jednak ich znajomości aby móc odpowiednio zabezpieczyć i zapewnić poprawne działanie więzień. Omówione przez mnie ustawienia są wykorzystywane w jail-manager

2.5.1 Gospodarz

Plik konfiguracyjny definiujący reguły montowania urządzeń wirtualnych powinien zawierać `devfs_ruleset=5` z fragmentem "add path 'bpf*' unhide"

Listing 14: devfs.rules (s9.volt.zet.pw.edu.pl/FreeBSD)

```
1 # cat /etc/defaults/devfs.rules
2 ...
3 [devfsrules_jail_vnet=5]
4 add include $devfsrules_hide_all
5 add include $devfsrules_unhide_basic
6 add include $devfsrules_unhide_login
7 add include $devfsrules_jail
8 add path pf unhide
9 add path 'bpf*' unhide
```

Plik konfiguracyjny `rc.conf` musi umożliwiać kreację więzień

Listing 15: rc.conf (s9.volt.zet.pw.edu.pl/FreeBSD)

```
1 # cat /etc/rc.conf
2 ...
3 jail_enable="YES"
```

2.5.2 Pliki konfiguracyjne więzień

Więzienia mogą być tworzone poprzez zdefiniowanie atrybutów w lini poleceń lub poprzez uruchomienie z pliku konfiguracyjnego. Pliki konfiguracyjne od wersji 13.1 domyślnie mogą być tworzone w `/etc/jail.conf.d/`, możliwe jest również zamieszczenie konfiguracji w pliku `/etc/jail.conf` dzięki czemu nie jest wymagane specyfikowanie pliku przy tworzeniu, takie rozwiązanie jest jednak nie wygodne przy dużej liczbie więzień.

- `enforce_statfs` - mówi o widoczności punktów montowania z systemu gospodarza. W zależności od zedniowanej wartości: 2-niewodczne; 1-tylko wykorzystywane zasoby; 0-wszystko
- `allow.raw_socket` - umożliwia działanie programów wymagających surowego gniazda np. ping
- `allow.mount` - określa pozwolenia na montowanie zasobów wewnątrz więzienia
- `path` - ścieżka do systemu plików
- `devfs_rulset` - określa reguły przypisaną do więzienia z pliku konfiguracyjnego definiującego reguły montowania urządzeń wirtualnych
- `vnet` - umożliwia obsługę wirtualnego stosu sieciowego

Listing 16: Domyślny szablon z jail-manager

```
1 $JAIL_NAME {
2     path="$JAIL_DIR";
3     host.hostname="$JAIL_NAME";
4     allow.mount;
5     mount.devfs;
6     devfs_ruleset="5";
7     mount.fstab="/etc/fstab_jail.${JAIL_NAME}";
8     vnet;
9     enforce_statfs=1;
10    vnet.interface="epb_${JAIL_NAME}";
11    exec.start="/bin/sh /etc/rc.conf";
12    exec.stop="/bin/sh /etc/rc.shutdown";
13    exec.poststart="jexec ${JAIL_NAME} dhclient -b epb_${JAIL_NAME} && jexec ${JAIL_NAME}
service sshd start";
14    exec.poststop="jexec ${JAIL_NAME} service sshd stop";
15    exec.clean;
16 }
```

Listing 17: Uruchamianie więzienia z pliku konfiguracyjnego

```
1 # jail -i -f "/etc/jail.conf.d/${name}.conf" -c
```


2.6 Program jail-manager

Na program składają się skrypty dostępne w załącznikach.

2.6.1 Cel

Automatyzacja procesu zarządzania i tworzenia więzień.

2.6.2 Zależności

Skrypty korzystają z: zasobów serwera dysków NAS - nas2, w procesie instalacji systemu bazowego wykorzystuje dostępne archiwa z wersjami FreeBSD; pakietu FreeBSD jail; pakietu ZFS

2.6.3 Środowisko

Skrypty napisane są w POSIX shell-u, działają w interpreterze poleceń zsh wykonywane z uprawnieniami roota, w systemie operacyjnym FreeBSD.

2.6.4 Scenariusz 1 - tworzenie systemu bazowego

Tworzy RAM dysk o pojemności 10g i system bazowy z system operacyjnym FreeBSD w wersji 14.0. Pula zostaje zamontowana w /jails. Podczas instalacji zostaje również utworzony mostek sieciowy jail_bridge.

Listing 18: Konfiguracja środowiska i stworzenie systemu bazowego (s9.volt.zet.pw.edu.pl/FreeBSD)

```
1 # jail-manager -i
2 ramdisk ready
3 jail_bridge
4 bridge ready
5 Preparing base jail...
6 base jail installed
7 Jails enviroment ready to use
8 sudo jail-manager -i 19.70s user 8.03s system 88% cpu 31.431 total
9 # df -h
10 Filesystem                Size      Used    Avail Capacity  Mounted on
11 /dev/md0                   484M      217M    228M      49%        /
12 devfs                      1.0K        0B     1.0K        0%      /dev
13 tmpfs                      3.4G       12K    3.4G        0%      /tmp
14 tmpfs                      3.5G       58M    3.4G        2%      /var
15 tmpfs                      4.7G      1.3G    3.4G       27%      /usr
16 ftp:/pub                   3.6T      2.7T   920G       75%      /pub
17 ftp:/usr/local/zetis       934G       14G   920G        1%  /usr/local/zetis
18 jails_pool                  8.6G       24K    8.6G        0%      /jails
19 jails_pool/base_jail        9.2G      611M    8.6G        6%  /jails/base_jail
20 # ifconfig jail_bridge
21 jail_bridge: flags=1008843<UP,BROADCAST,RUNNING,SIMPLEX,MULTICAST,LOWER_UP> metric 0 mtu
    1500
22     options=0<>
23     ether 58:9c:fc:10:ff:9f
24     id 00:00:00:00:00:00 priority 32768 hellotime 2 fwddelay 15
25     maxage 20 holdcnt 6 proto rstp maxaddr 2000 timeout 1200
26     root id 00:00:00:00:00:00 priority 32768 ifcost 0 port 0
27     member: em1 flags=143<LEARNING,DISCOVER,AUTOEDGE,AUTOPTP>
28             ifmaxaddr 0 port 2 priority 128 path cost 20000
29     groups: bridge
30     nd6 options=9<PERFORMNUD,IFDISABLED>
```

Listing 19: Snapshoty systemu bazowego

```
1 # zfs list -t snapshot
2 NAME                                USED    AVAIL    REFER  MOUNTPOINT
3 jails_pool/base_jail@clean           22K     -       24K    -
4 jails_pool/base_jail@template        0B      -       611M    -
```

2.6.5 Scenariusz 2 - zarządzanie więzieniami

Zaprezentowane akcje mogą być uruchomiona podając ścieżkę do pliku zawierającego listę nazw więzień. Zatrzymanie (`-stop`), usuwanie (`-destroy`) i startowanie (`-start`) może być użyte z paremtrzem `all`

Listing 20: Tworzenie więzienia

```
1 # jail-manager --create jail1
2 jail 'jail1' created
3 >successfully created: 1
4 >failed to create: 0
5 # zfs list
6 NAME                                USED    AVAIL    REFER    MOUNTPOINT
7 jails_pool                          611M    8.61G    25K      /jails
8 jails_pool/base_jail                611M    8.61G    611M     /jails/base_jail
9 jails_pool/jail1                    332K    8.61G    611M     /jails/jail1
```

`zfs list` Pokazuje jak mało pamięci dzięki zastosowaniu systemu bazowego zajmują kolejno tworzone więzienia

Listing 21: Logowanie do więzienia (`s9.volt.zet.pw.edu.pl/FreeBSD`)

```
1 # jexec jail1 ifconfig
2 lo0: flags=8008<LOOPBACK,MULTICAST> metric 0 mtu 16384
3     options=680003<RXCSUM,TXCSUM,LINKSTATE,RXCSUM_IPV6,TXCSUM_IPV6>
4     groups: lo
5     nd6 options=21<PERFORMNUD,AUTO_LINKLOCAL>
6 epb_jail1: flags=8843<UP,BROADCAST,RUNNING,SIMPLEX,MULTICAST> metric 0 mtu 1500
7     options=8<VLAN_MTU>
8     ether 02:7c:0a:ee:b8:0b
9     inet 10.146.90.162 netmask 0xffff0000 broadcast 10.146.255.255
10    groups: epair
11    media: Ethernet 10Gbase-T (10Gbase-T <full-duplex>)
12    status: active
13    nd6 options=29<PERFORMNUD,IFDISABLED,AUTO_LINKLOCAL>
14 # ssh skibas@10.146.90.162
15 The authenticity of host '10.146.90.162 (10.146.90.162)' can't be established.
16 ED25519 key fingerprint is SHA256:+f35Vv47o58kEJTbuKvDyilU+19+vds0wFXXK26Ej+k.
17 This key is not known by any other names.
18 Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
19 Warning: Permanently added '10.146.90.162' (ED25519) to the list of known hosts.
20 (skibas@10.146.90.162) Password for skibas@jail1:
21 % uname -a
22 FreeBSD jail1 14.0-CURRENT FreeBSD 14.0-CURRENT #0 main-n263302-4f2cc73f34eb: Thu Jun 1
    08:07:09 UTC 2023      root@releng1.nyi.freebsd.org:/usr/obj/usr/src/amd64.amd64/sys/
    GENERIC amd64
```

Listing 22: Start stop usuń (`s9.volt.zet.pw.edu.pl/FreeBSD`)

```
1 # jail-manager --stop all
2 jail 'jail1' stopped
3 >successfully stopped: 1
4 >failed to stop: 0
5 # jail-manager --start jail1
6 jail 'jail1' started
7 >successfully started: 1
8 >failed to start: 0
9 # jail-manager --destroy jail1
10 jail 'jail1' destroyed
11 >successfully destroyed: 1
12 >failed to destroy: 0
```

Listing 23: Wyświetlanie listy (`s9.volt.zet.pw.edu.pl/FreeBSD`)

```
1 # jail-manager -l
2 Jail          status          ipv4
3 jail1         working         10.146.90.150
4 jail2         working         10.146.90.83
```

2.6.6 Scenariusz 3 - zarządzanie systemem bazowym

Skrypty udostępniają trzy możliwe operacje na systemie bazowym:

Reinatalacja - usuwanie całe środowisko i wszystkie więzienia. Następnie ponownie instaluje system

Listing 24: (s9.volt.zet.pw.edu.pl/FreeBSD)

```
1 # jail-manager --base -r
2 Jails environment deleted
3 ramdisk ready
4 jail_bridge
5 bridge ready
6 Preparing base jail...
7 base jail installed
8 Jails environment ready to use
9 sudo jail-manager --base -r 19.87s user 5.12s system 94% cpu 26.336 total
10 # zfs list
11 NAME                USED    AVAIL    REFER    MOUNTPOINT
12 jails_pool            611M    8.61G    24K      /jails
13 jails_pool/base_jail  611M    8.61G    611M     /jails/base_jail
```

Edycja - daje możliwość ręcznej edycji systemów systemu bazowego, następnie tworzy migawkę co pozwala w łatwy sposób przetestować zmiany i ewentualnie cofnąć je

Listing 25: (s9.volt.zet.pw.edu.pl/FreeBSD)

```
1 # jail-manager --base -u
2 After leaving (exit) snapshot will be created and base jail will be updated
3 root@s9:/ # vi /etc/rc.conf
4 root@s9:/ # exit
5 exit
6 base jail updated
```

Porót do migawki

Listing 26: (s9.volt.zet.pw.edu.pl/FreeBSD)

```
1 # jail-manager --base -s
2 Choose snapshot to rollback
3 NAME                USED    AVAIL    REFER    MOUNTPOINT
4 jails_pool/base_jail@clean      22K      -      24K      -
5 jails_pool/base_jail@2023-06-8_20:30:24 191K      -      611M      -
6 jails_pool/base_jail@template    0B      -      611M      -
7 jails_pool/base_jail@2023-06-8_20:30:24
8 # zfs list -t snapshot
9 NAME                USED    AVAIL    REFER    MOUNTPOINT
10 jails_pool/base_jail@clean      22K      -      24K      -
11 jails_pool/base_jail@template    1K      -      611M      -
```

2.6.7 Scenariusz 4 - więzienia z volt

Jeżeli użytkownik wykonujący skrypty ma konto z odpowiednimi uprawnieniami na volcie jest możliwe załączenie użytkowników i stworzenie analogicznych więzień z użytkownikami i folderami domowymi w /jails/jail_name/home/jail_name

Listing 27: (s9.volt.zet.pw.edu.pl/FreeBSD)

```
1 # jail-manager --volt
2 (skibas@vol.zet.pw.edu.pl) Password for skibas@volt.zet.pw.edu.pl:
3 jail 'ankiersm' created
4 jail 'balam' created
5 jail 'banaszmi' created
6 jail 'bankowj1' created
7 ...
8 # zfs list
9 NAME                USED    AVAIL    REFER    MOUNTPOINT
10 jails_pool            626M    8.59G    35K      /jails
```

```

11 jails_pool/ankiersm      364K  8.59G  611M  /jails/ankiersm
12 jails_pool/balam        363K  8.59G  611M  /jails/balam
13 jails_pool/banaszm1     364K  8.59G  611M  /jails/banaszm1
14 jails_pool/bankowj1     364K  8.59G  611M  /jails/bankowj1
15 ...

```

3 Źródła

[1] <https://docs.freebsd.org/pl/books/handbook/>

[2] Marcin Michta, "Parawirtualizacja w środowisku FreeBSD", 2014

[3] <https://issue.freebsdoundation.org/publication/?m=33057&i=645584&view=contentsBrowser&ver=html5> - FreeBSD journal

[4] <https://bsdrp.net/BSDRP>

[5] <https://wiki.freebsd.org/>

[6] <https://download.freebsd.org/releases/>

[7] <ftp://nas2.zet.pw.edu.pl/>

4 Skrypty

[8] jail-manager

[9] jail-base

[10] jail-bridge

[11] jail-create

[12] jail-ramdisk